C Programming

NovoNex Delta



Strings: Character Arrays & Null Character (\0)

- **What is a String?:** In C, strings are 1D character arrays terminated by a special null character '\0' (ASCII 0).
- **Memory Layout:** String literal "HELLO" occupies 6 contiguous bytes in RAM: ['H', 'E', 'L', 'L', 'O', '\0'].
- **Declaration & Init:** char s[] = "Hello"; or char s[6] = {'H','e','l','l','o','\0'};
- **Safe Input (fgets vs scanf):** scanf("%s") stops at spaces and overflows; fgets(s, sizeof(s), stdin) is safe!
- **String Traversal:** Traverse sequentially using a loop until str[i] == '\0' is encountered.

```c
// Safe String Input, Traversal & Output
char name[50];
printf("Enter full name: ");
fgets(name, sizeof(name), stdin);
int len = 0;
while (name[len] != '\0' && name[len] != '\n') {
    len++;
}
printf("Length = %d, Greeting: %s", len, name);
```

⚠️ Buffer Overflow: Always size array +1 for '\0'! Never use unsafe gets()!

✏️ Live Trace & Explanation Board

[Teacher's Note: Draw RAM memory boxes for 'B','O','O','K','\0' with addresses (0x100..0x104) and show why printf stops precisely at '\0']

Standard String Library Functions (<string.h>)

- **strlen(str):** Returns the count of characters strictly before '\0'. (Time complexity: O(N)).
- **strcpy(dest, src):** Copies source string into destination. dest must have sufficient allocated memory!
- **strcat(dest, src):** Concatenates (appends) src to the end of dest, overwriting dest's '\0'.
- **strcmp(s1, s2):** Lexicographical comparison: returns 0 if equal, <0 if s1 < s2, and >0 if s1 > s2.
- **Two-Pointer Reverse:** Reverse string in-place by swapping str[left] and str[right] until pointers meet.

```c
            // Built-in String Operations (<string.h>)
char s1[30] = "Novo", s2[] = "Nex";
strcat(s1, s2);             // s1 becomes "NovoNex"
int cmp = strcmp(s1, "NovoNex");
if (cmp == 0) {             // 0 means exact MATCH!
    printf("Matched! Length = %zu\n", strlen(s1));
}
```

💡 strcmp Trap: strcmp returns 0 on MATCH! Condition: if (strcmp(a, b) == 0) means TRUE!

✏️ Live Trace & Explanation Board

[Teacher's Note: Live trace strcmp letter-by-letter ASCII comparison ('A'=65 vs 'B'=66) and demonstrate two-pointer reverse logic]

Modular Programming & Function Architecture

- **Why Functions?:** Break complex programs into reusable, isolated, testable modules (DRY principle).
- **3 Essential Components:** Every C function requires:
- **1. Prototype Declaration:** Informs compiler of name, return type & parameters before main().
- **2. Function Definition:** The actual implementation block containing executable instructions.
- **3. Function Call:** Invoking the function by passing argument values to execute its body.
- **Return Types:** Returns computed result (int, double, etc.) or void if no value is returned.

```c
          // Function Prototype, Definition & Invocation
int add(int a, int b);       // 1. Prototype Declaration
int main() {
    int result = add(15, 25); // 3. Function Call
    printf("Sum = %d\n", result);
    return 0;
}
int add(int a, int b) {      // 2. Function Definition
    return a + b;
}
```

💡 **Prototype Rule: Placing definition after main() requires a prototype before main()!**

✏️ Live Trace & Explanation Board

[Teacher's Note: Trace instruction pointer jump from main() to add(), return value in CPU register EAX, and jump back to main()]

Variable Scope, Stack Frames & Pass by Value

- **Pass by Value:** C functions receive COPIES of argument values. Modifying parameters does NOT change caller variables!
- **Stack Frame Allocation:** Each function call allocates an isolated stack frame in RAM for its local variables.
- **Local vs Global Scope:** Local variables exist only inside their { } block; Globals exist across the entire program.
- **Stack Frame Destruction:** Returning from a function pops and destroys its stack frame and local variables instantly.
- **static Variables:** Retain their persistent value across multiple function calls without re-initializing!

```c
            // Pass-by-Value Demo & Static Persistence
void tryModify(int x) { x = 999; } // Modifies only local copy
void counter() {
    static int count = 0;   // Retains value across calls
    count++;
    printf("Call count: %d\n", count);
}
// In main: int val = 10; tryModify(val); -> val is STILL 10!
```

⚠️ **Pass by Value Trap: Caller's variable remains unmodified! Pointers are required to mutate caller memory!**

✏️ Live Trace & Explanation Board

[Teacher's Note: Draw RAM Call Stack with separate frames for main() vs tryModify() to prove variable memory isolation]

Pointers: Direct Memory Access & Addressing

- **What is a Pointer?:** A special variable that stores the physical RAM memory address (0x7ffd...) of another variable.
- **Address-Of Operator (&):** Retrieves the exact memory address where a variable is stored in RAM.
- **Dereference Operator (*):** Accesses or modifies the value residing at the pointed address (*ptr = val).
- **Pointer Declaration:** int *ptr; indicates ptr points to an int. (Occupies 8 bytes on 64-bit OS).
- **NULL Pointer:** Initializing pointer to NULL (ptr = NULL;) ensures it points safely to address 0x0.

```c
// Pointer Declaration, Addressing & Dereferencing
int x = 42;
int *ptr = &x;     // ptr stores the memory address of x
printf("Value of x: %d\n", *ptr);  // 42 (Dereference)
printf("Address of x: %p\n", ptr); // Hex RAM address
*ptr = 100;        // Modifies x indirectly via pointer!
printf("New x = %d\n", x);        // Prints 100
```

⚠️ **Wild Pointer: Dereferencing an uninitialized pointer causes Segmentation Fault / Crash!**

✏️ Live Trace & Explanation Board

[Teacher's Note: Draw variable x box (val=42, addr=0x1000) and ptr box (val=0x1000, addr=0x2000) with arrow pointing from ptr to x]

Pass by Reference & Pointer Arithmetic

- **Pass by Reference (via Pointers):** Passing memory addresses allows a function to mutate caller variables directly!
- **Classic Swap Function:** void swap(int *a, int *b) modifies caller values directly in their original memory slots.
- **Pointer Arithmetic:** ptr + 1 advances the address by sizeof(type) bytes (e.g. +4 bytes for int), not 1 raw byte!
- **Array-Pointer Relationship:** Array name arr decays into a constant pointer to its first element (&arr[0]).
- **Equivalence Rule:** *(arr + i) is 100% equivalent to arr[i].

```c
// Working Swap & Array Traversal with Pointers
void swap(int *a, int *b) {
    int temp = *a; *a = *b; *b = temp;
}
int arr[3] = {10, 20, 30};
int *p = arr;
for (int i = 0; i < 3; i++) {
    printf("%d ", *(p + i)); // Pointer offset traversal
}
```

💡 **Universal Equivalence: arr[i] == *(arr + i) == *(i + arr) == i[arr]! All resolve to identical offset!**

✏️ Live Trace & Explanation Board

[Teacher's Note: Trace swap(&x, &y) memory links between stack frames; show ptr++ jumping +4 bytes across contiguous array cells]

Recursion: Self-Referential Functions & Base Cases

- **What is Recursion?:** A problem-solving technique where a function calls itself to solve smaller subproblems.
- **2 Vital Components:** Every valid recursive function MUST possess:
- **1. Base Case:** The terminating condition that returns immediately without further recursive calls.
- **2. Recursive Step:** Progresses toward the base case by calling itself with reduced input.
- **Call Stack Accumulation:** Each recursive invocation pushes a fresh stack frame onto RAM, suspending the caller.
- **Unwinding Phase:** When base case triggers, return values propagate back up the stack to compute final result.

```c
// Factorial Calculation using Recursion
int factorial(int n) {
    if (n <= 1) {          // 1. Base Case
        return 1;
    }
    return n * factorial(n - 1); // 2. Recursive Step
}
// In main: factorial(4) computes: 4 * 3 * 2 * 1 = 24
```

⚠️ **Stack Overflow: Omitting a base case causes infinite recursion until the call stack memory is exhausted!**

✏️ Live Trace & Explanation Board

[Teacher's Note: Draw Call Stack growing upwards: fact(4)->fact(3)->fact(2)->fact(1), then draw downward return resolution]

Tree Recursion: Execution Branching & Complexity

- **Linear vs Tree Recursion:** Linear recursion calls itself once; Tree recursion branches into 2 or more self-calls per step.
- **Fibonacci Tree Model:** fib(n) = fib(n-1) + fib(n-2) produces an exponential binary tree of subproblems.
- **Time Complexity O(2^N):** Naive tree recursion recalculates identical subproblems redundantly (e.g. fib(2) multiple times).
- **Space Complexity:** Auxiliary stack space equals maximum tree depth O(N).
- **Prime Use Cases:** Divide & Conquer (Merge Sort, Quick Sort), Tree/Graph traversals, and Backtracking algorithms.

```c
                // Tree Recursion: Fibonacci Sequence
int fib(int n) {
    if (n <= 0) return 0;  // Base case 1
    if (n == 1) return 1;  // Base case 2
    return fib(n - 1) + fib(n - 2); // 2 Branches!
}
// fib(4) expands into binary tree evaluating:
// fib(3) + fib(2) -> [fib(2)+fib(1)] + [fib(1)+fib(0)]
```

💡 **Dynamic Programming Insight: Redundant subproblem recomputation in tree recursion is solved via DP (Memoization)!**

✏️ Live Trace & Explanation Board

[Teacher's Note: Draw full binary call tree for fib(4), circle duplicate fib(2) calculations, and trace recursive call timeline]

Dynamic Memory Allocation: Stack vs Heap Memory

- **Why DMA?:** Fixed static arrays waste space or overflow. DMA allocates arbitrary memory at runtime based on user demand!
- **Stack vs Heap:** Stack is fast and auto-cleared; Heap is huge, flexible, and persists until manually deallocated.
- **malloc(size_bytes):** Allocates contiguous uninitialized bytes from Heap (contains random garbage values!).
- **calloc(n, size):** Allocates memory for n elements and initializes EVERY single byte to ZERO (0).
- **void* & Typecasting:** Both return a generic pointer void*; cast to typed pointer (e.g. (int*)malloc(...)).
- **Allocation Failure Check:** Returns NULL if system RAM is exhausted. Always verify pointer validity!

```c
    // Dynamic Array Allocation with malloc & calloc (<stdlib.h>)
int n = 5;
int *arr = (int*)malloc(n * sizeof(int));
if (arr == NULL) {
    printf("Heap allocation failed!\n");
    return 1;
}
for (int i = 0; i < n; i++) arr[i] = (i + 1) * 10;
```

⚠️ **NULL Pointer Check: Always verify ptr != NULL immediately after malloc/calloc to avoid instant crashes!**

✏️ Live Trace & Explanation Board

[Teacher's Note: Sketch 4 RAM Segments: Code -> Data/BSS -> Heap (grows UP) -> Stack (grows DOWN); trace heap allocation pointer]

DMA Lifecycle: Resizing, Deallocation & Safety

- **free(ptr):** Deallocates heap memory back to the Operating System. Leaves pointer variable intact!
- **Memory Leak:** Failing to free() allocated heap memory causes program RAM consumption to grow unbounded.
- **Dangling Pointer Trap:** A pointer still referencing memory that has already been freed! Fix: assign ptr = NULL;.
- **realloc(ptr, new_size):** Dynamically resizes an existing heap block:
- **- Mechanism:** Expands or shrinks in-place if possible, or moves block to new heap address and copies contents.
- **Double Free Bug:** Calling free(ptr) twice on the same address triggers heap corruption / crashes.

```c
            // Resizing with realloc & Safe Deallocation
int *temp = (int*)realloc(arr, 10 * sizeof(int));
if (temp != NULL) {
    arr = temp;           // Update base pointer safely
}
// After finishing usage:
free(arr);                // 1. Release heap memory
arr = NULL;               // 2. Prevent dangling pointer!
```

💡 **DMA Golden Rule: Every malloc/calloc MUST have exactly one matching free(ptr); followed by ptr = NULL;!**

✏️ Live Trace & Explanation Board

[Teacher's Note: Illustrate Dangling Pointer vs NULL pointer in memory, and trace realloc memory relocation with copy behavior]

Structures (struct): User-Defined Composite Types

- **What is a struct?:** A user-defined data type grouping related variables of DIFFERENT types under a single unified name.
- **Array vs Structure:** Arrays are homogeneous (all same type); Structures are heterogeneous (int, float, char[]).
- **Struct Declaration:** struct Student { int id; char name[20]; float gpa; }; defines the blueprint.
- **Dot (.) Operator:** Direct member access syntax used to read or modify fields: s1.id = 101;.
- **Memory Layout & Padding:** Compiler inserts unused padding bytes to align data fields with CPU word boundaries!
- **Initialization:** Direct assignment struct Student s1 = {101, "Ali", 3.9}; assigns fields in declared order.

```c
// Structure Declaration, Instantiation & Member Access
struct Student {
    int id;
    char name[30];
    float gpa;
};
struct Student s1 = {101, "Shohan", 3.85};
printf("ID: %d | Name: %s | GPA: %.2f\n", s1.id, s1.name, s1.gpa);
```

⚠️ **Structure Padding: sizeof(struct) > sum of members due to CPU word alignment padding!**

✏️ Live Trace & Explanation Board

[Teacher's Note: Draw memory map of struct Student showing 4 bytes (int), 30 bytes (char), padding bytes, and 4 bytes (float)]

Advanced Structures: Pointers, Arrays & typedef

- **typedef Keyword:** Eliminates repetitive struct keyword, creating clean type alias: typedef struct { ... } Student;.
- **Array of Structures:** Manages collections of records: Student batch[100]; batch[i].gpa = 3.9;.
- **Pointer to Structure:** Student *ptr = &s1; stores address of struct in RAM.
- **Arrow (->) Operator:** Shorthand to access fields through pointer: ptr->name is 100% equivalent to (*ptr).name.
- **Function Passing:** Passing by pointer (void print(const Student *s)) avoids copying entire struct block (super fast!).
- **Nested Structures:** Structs can embed other structs as members (e.g. Date dob; inside Student).

```c
// typedef, Struct Pointer, Arrow Operator & Functions
typedef struct {
    int id; float gpa;
} Student;
void display(const Student *s) {
    printf("ID: %d, GPA: %.2f\n", s->id, s->gpa); // Arrow ->
}
Student s1 = {201, 3.95};
display(&s1); // Pass by pointer (Zero copy overhead)
```

💡 **Efficiency Rule: Always pass structs to functions via pointer (const Type *ptr) to eliminate memory copying overhead!**

✏️ Live Trace & Explanation Board

[Teacher's Note: Diagram struct memory block and show pointer ptr with arrow (->) directly targeting member offset inside]

Thank You

Happy Coding with C & NovoNex Delta!

